

3. Un carré de 8 × 8

Une case de l'écran est un carré de 8 × 8 pixels : on en pose un.

1. Ce qu'il faut comprendre

Chaque case de l'écran est un **carré de 8 × 8 pixels**, qu'on appelle une **tuile**. Une lettre est une tuile ; ce carré aussi.

Tuile `CARRE = { ... };` dessine la tuile : **8 lignes de 8 chiffres**, un chiffre par pixel. `3` est la nuance la plus foncée, `0` la plus claire.

`poser(colonne, ligne, CARRE)` pose la tuile dans une case. L'écran fait 20 × 18 cases, soit **160 × 144 pixels**.

Les lignes `#include <...>`, en haut, disent ce que le programme emploie de la console : `#include <Tuile>` pour dessiner une tuile, `#include <poser>` pour la poser. Aucune fonction de la console n'est là d'office : chacune prend de la **place dans la cartouche**, et ne s'emploie que si on l'**inclut**, par son nom écrit comme dans le programme. Sans la ligne, le compilateur refuse, et dit laquelle écrire. Une ligne de trop ne coûte rien.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **175 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	13 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	16 o	← CopierTuiles — 0 lettre sur 43, et 1 tuile dessinée

Lettres gravées : aucune

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>poser()</code> × 1	6 o	sur place, dans <code>main()</code>
<code>image()</code> × 1	3 o	sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : pose un carré noir de 8 × 8 pixels au milieu de l'écran.
//
// « // » commence un commentaire : la console ne le lit pas.
//
// Une TUILE est un petit dessin de 8 × 8 pixels.
// Chaque texte entre guillemets est une rangée de 8 pixels.
// Chaque chiffre est un pixel : 0 = le plus clair, 1, 2, 3 = le plus foncé.

// Un carré de 8 × 8 pixels : 8 lignes de 8 chiffres.
// 3 = pixel le plus foncé.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <poser>    // pose une tuile sur une case du fond
```

2. UN DESSIN — CARRE

```
Tuile CARRE = {
  "33333333",
  "33333333",
  "33333333",
  "33333333",
  "33333333",
  "33333333",
  "33333333",
  "33333333",
  "33333333",
  "33333333",
};
```



à l'écran

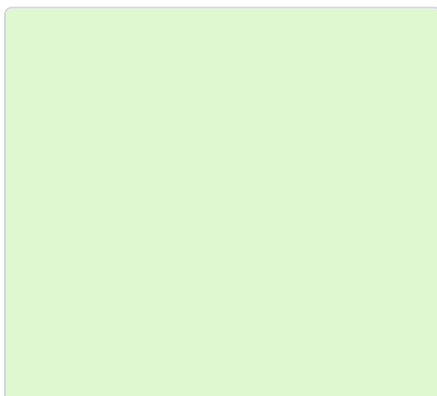
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {           // main = « principal » : le jeu commence ICI.
  poser(9, 8, CARRE);  // colonne 9, ligne 8 : le milieu de l'écran
                      // L'écran a 20 colonnes (0 à 19) et 18 lignes (0 à 17).
                      // Le « ; » termine l'instruction, comme un point en fin de phrase.

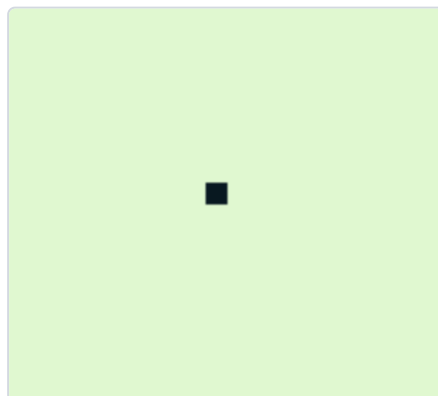
  while (true) image(); // garde l'écran affiché :
                      // while (true) répète pour toujours ; image() attend l'image suivante.
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un carré noir de 8×8 au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le carré est posé en (9, 8)
- ✓ il est seul : la case d'à côté est vide
- ✓ l'écran est allumé

Ensuite : **4. Écrire à plusieurs endroits** — chapitre 1 : Les tout premiers pas. · [gameboy3](#) — écrire une cartouche Game Boy en C++